P1301R0
nodiscard should have a reason

Published Proposal, 2018-10-07

Authors:

JeanHeyd Meneide

Isabella Muerte

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/ThePhD/future_cxx/blob/master/papers/source/d1301.bs

Current:

https://rawgit.com/ThePhD/future_cxx/papers/d1301.html

Reply To:

@slurpsmadrips

Abstract

This paper proposes allowing an explanatory string token for use with the nodiscard attribute.

Table of Contents

§ 1. Revision History

§ 1.1. Revision 0

Initial release.

§ 2. Motivation

The `[[nodiscard]]` attribute has helped prevent a serious class of software bugs, but sometimes it is hard to communicate exactly why a function is marked as `[[nodiscard]]`.

This paper adds an addendum to allow a person to add a string attribute token to let someone provide a small reasoning or reminder for why a function has been marked `[[nodiscard("Please do not leak this memory!")]]`.

§ 3. Design Considerations

This paper is an enhancement of a preexisting feature to help programmers provide clarity with their code. Anything that makes the implementation warn or error should also provide some reasoning or perhaps point users to a knowledge base or similar to have any questions they have about the reason for the `nodiscard` attribute answered.

The design is very simple and follows the lead of the `deprecated` attribute. We propose allowing a string literal to be passed as an attribute argument clause, allowing for `[[nodiscard("You must use the returned token with api::foobar")]]`. The key here is that there are some `nodiscard` attributes that have different kinds of "severity" versus others. That is, the cost of discarding `vector::empty`'s return value is probably of slightly less concern than

`unique_ptr::release`: one might manifest as a possible bug, the other is a downright memory leak.

Adding a reason to `nodiscard` allows implementers of the standard library, library developers, and application writers to benefit from a more clear and concise error beyond "Please do not discard this return value". This makes it easier for developers to understand the intent of the code they are using.

§ 4. Wording

All wording is relative to [n4762].

§ 4.1. Intent

The intent of the wording is to let a programmer specify a string literal that the compiler can use in their warning / error display for, e.g. `[[nodiscard("Please check the status variable")]]`.

§ 4.2. Feature Test Macro

`nodiscard` already has an attribute value in Table 15. If this proposal is accepted, then per the standard's reasoning the value of `nodiscard` should be increased to some greater value (such as `201811L`).

§ 4.3. Proposed Wording

Modify §14.1 [**cpp.cond**]'s **Table 15**:

> | **Attribute** | **Value** |
> |---|---|
> | nodiscard | ~~201603L~~ 201811L |

Append the following to §9.11.9 Nodiscard attribute [**dcl.attr.nodiscard**]'s clause 1:

> [1] The *attribute-token* nodiscard may be applied to the declarator-id in a function declaration or to the declaration of a class or enumeration. It shall appear at most once in each attribute-list and no *attribute argument-clause* shall be present. An attribute-argument-clause may be present and, if present, it shall have the form:
>
> (string-literal)
>
> [Note: The string-literal in the attribute-argument-clause could be used to explain the rationale for why the entity must not be discarded. — end note]

§ References

§ Informative References

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4762 - Working Draft, Standard for Programming Language C++. May 7th, 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf